COMP 442
Compiler Design

Assignment 2: Syntactic Analyzer

Writer:
Théo Bary (ID: 40348931)

Instructor: Dr. Joey Paquet
joey.paquet@concordia.ca

Date Due: Sunday, 15 Feb 2026, 11:59 PM

"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"

Section 1 - Grammar: 2
Section 2 - FIRST and FOLLOW sets: 5
Section 3 - Design: 6
Section 4 - Use of tools: 7

Section 1 - Grammar:

Grammar transformation was done by analyzing FLACI's LL(1) check and resolving the rule violations.

Original BNF	My grammar transformed into LL(1)

```
<START>        ::= <prog>
<prog>         ::= {{<classDecl>}} {{<funcDef>}} 'main' <funcBody>
<classDecl>    ::= 'class' 'id' [['inherits' 'id' {{',' 'id'}}]] '{' {{<visibility> <memberDecl>}} '}' ';'
<visibility>   ::= 'public' | 'private'
<memberDecl>   ::= <funcDecl> | <varDecl>
<funcDecl>     ::= 'id' '(' <fParams> ')' ':' <type> ';' | 'id' '(' <fParams> ')' ':' 'void' ';'
<funcHead>     ::= [['id' 'sr']] 'id' '(' <fParams> ')' ':' <type> | [['id' 'sr']] 'id' '(' <fParams> ')' ':' 'void'
<funcDef>      ::= <funcHead> <funcBody>
<funcBody>     ::= [[ 'local' {{<varDecl>}} ]] 'do' {{<statement>}} 'end'
<varDecl>      ::= <type> 'id' {{<arraySize>}} ';'
<statement>    ::= <assignStat> ';'
               | 'if'     '(' <relExpr> ')' 'then' <statBlock> 'else' <statBlock> ';'
               | 'while'  '(' <relExpr> ')' <statBlock> ';'
               | 'read'   '(' <variable> ')' ';'
               | 'write'  '(' <expr> ')' ';'
               | 'return' '(' <expr> ')' ';'
               | <functionCall> ';'
<assignStat>   ::= <variable> <assignOp> <expr>
<statBlock>    ::= 'do' {{<statement>}} 'end' | <statement> | EPSILON
<expr>         ::= <arithExpr> | <relExpr>
<relExpr>      ::= <arithExpr> <relOp> <arithExpr>
<arithExpr>    ::= <arithExpr> <addOp> <term> | <term>
<sign>         ::= '+' | '-'
<term>         ::= <term> <multOp> <factor> | <factor>
<factor>       ::= <variable>
               | <functionCall>
               | 'intNum' | 'floatNum'
               | '(' <arithExpr> ')'
               | 'not' <factor>
               | <sign> <factor>
<variable>      ::= {{<idnest>}} 'id' {{<indice>}}
<functionCall> ::= {{<idnest>}} 'id' '(' <aParams> ')'
<idnest>       ::= 'id' {{<indice>}} '.'
               | 'id' '(' <aParams> ')' '.'
<indice>       ::= '[' <arithExpr> ']'
<arraySize>    ::= '[' 'intNum' ']' | '[' ']'
<type>         ::= 'integer' | 'float' | 'id'

<fParams>      ::= <type> 'id' {{<arraySize>}} {{<fParamsTail>}} | EPSILON
<aParams>      ::= <expr> {{<aParamsTail>}} | EPSILON
<fParamsTail>  ::= ',' <type> 'id' {{<arraySize>}}
<aParamsTail>  ::= ',' <expr>
<assignOp>     ::= '='
<relOp>        ::= 'eq' | 'neq' | 'lt' | 'gt' | 'leq' | 'geq'
<addOp>        ::= '+' | '-' | 'or'
<multOp>       ::= '*' | '/' | 'and'
```

```
<START> ::= <prog>

<aParams> ::= <expr> <aParamsTail>
<aParams> ::= EPSILON

<aParamsTail> ::= ',' <expr> <aParamsTail>
<aParamsTail> ::= EPSILON

<addOp> ::= '+'
<addOp> ::= '-'
<addOp> ::= 'or'

<arithExpr> ::= <term> <arithExprTail>

<arithExprTail> ::= <addOp> <term> <arithExprTail>
<arithExprTail> ::= EPSILON

<arraySize> ::= '[' <arraySizeTail>

<arraySizeTail> ::= 'intNum' ']'
<arraySizeTail> ::= ']'

<assignOp> ::= '='

<classDecl> ::= 'class' 'id' <classInheritOpt> '{' <classMemberList> '}' ';'

<classInheritOpt> ::= 'inherits' 'id' <classInheritTail>
<classInheritOpt> ::= EPSILON

<classInheritTail> ::= ',' 'id' <classInheritTail>
<classInheritTail> ::= EPSILON

<classList> ::= <classDecl> <classList>
<classList> ::= EPSILON

<classMemberList> ::= <visibility> <memberDecl> <classMemberList>
<classMemberList> ::= EPSILON

<expr> ::= <arithExpr> <exprRelTail>

<exprRelTail> ::= <relOp> <arithExpr>
<exprRelTail> ::= EPSILON

<fParams> ::= <type> 'id' <fParamsArrayList> <fParamsTail>
<fParams> ::= EPSILON

<fParamsArrayList> ::= <arraySize> <fParamsArrayList>
<fParamsArrayList> ::= EPSILON

<fParamsTail> ::= ',' <type> 'id' <fParamsArrayList> <fParamsTail>
<fParamsTail> ::= EPSILON
```

```
<factor> ::= 'id' <postfixList>
<factor> ::= 'intNum'
<factor> ::= 'floatNum'
<factor> ::= '(' <arithExpr> ')'
<factor> ::= 'not' <factor>
<factor> ::= <sign> <factor>

<funcBody> ::= <localDeclOpt> 'do' <stmtList> 'end'

<funcDeclTail> ::= <type> ';'
<funcDeclTail> ::= 'void' ';'

<funcDef> ::= <funcHead> <funcBody>

<funcDefList> ::= <funcDef> <funcDefList>
<funcDefList> ::= EPSILON

<funcHead> ::= 'id' <funcHeadTail>

<funcHeadReturn> ::= <type>
<funcHeadReturn> ::= 'void'

<funcHeadTail> ::= 'sr' 'id' '(' <fParams> ')' ':' <funcHeadReturn>
<funcHeadTail> ::= '(' <fParams> ')' ':' <funcHeadReturn>

<indice> ::= '[' <arithExpr> ']'

<localDeclOpt> ::= 'local' <varDeclList>
<localDeclOpt> ::= EPSILON

<memberDecl> ::= <type_no_id> 'id' <varArrayList> ';'
<memberDecl> ::= 'id' <memberAfterId>

<memberAfterId> ::= 'id' <varArrayList> ';'
<memberAfterId> ::= '(' <fParams> ')' ':' <funcDeclTail>

<multOp> ::= '*'
<multOp> ::= '/'
<multOp> ::= 'and'

<postfix> ::= '(' <aParams> ')'
<postfix> ::= '[' <arithExpr> ']'
<postfix> ::= '.' 'id'

<postfixList> ::= <postfix> <postfixList>
<postfixList> ::= EPSILON

<postfixListNoCall> ::= <postfixNoCall> <postfixListNoCall>
<postfixListNoCall> ::= EPSILON
```

```
<prog> ::= <classList> <funcDefList> 'main' <funcBody>

<relOp> ::= 'eq'
<relOp> ::= 'neq'
<relOp> ::= 'lt'
<relOp> ::= 'gt'
<relOp> ::= 'leq'
<relOp> ::= 'geq'

<sign> ::= '+'
<sign> ::= '-'

<statBlock> ::= 'do' <stmtList> 'end'
<statBlock> ::= <statement>
<statBlock> ::= EPSILON

<statement> ::= 'id' <postfixList> <statementEnd>
<statement> ::= 'if' '(' <expr> ')' 'then' <statBlock> 'else' <statBlock> ';'
<statement> ::= 'while' '(' <expr> ')' <statBlock> ';'
<statement> ::= 'read' '(' <variable> ')' ';'
<statement> ::= 'write' '(' <expr> ')' ';'
<statement> ::= 'return' '(' <expr> ')' ';'

<statementEnd> ::= <assignOp> <expr> ';'
<statementEnd> ::= ';'

<stmtList> ::= <statement> <stmtList>
<stmtList> ::= EPSILON

<term> ::= <factor> <termTail>

<termTail> ::= <multOp> <factor> <termTail>
<termTail> ::= EPSILON

<type> ::= 'integer'
<type> ::= 'float'
<type> ::= 'id'

<type_no_id> ::= 'integer'
<type_no_id> ::= 'float'

<varArrayList> ::= <arraySize> <varArrayList>
<varArrayList> ::= EPSILON

<varDecl> ::= <type> 'id' <varArrayList> ';'

<varDeclList> ::= <varDecl> <varDeclList>
<varDeclList> ::= EPSILON

<variable> ::= 'id' <postfixListNoCall>
```

```
<visibility> ::= 'public'
<visibility> ::= 'private'
```

The size of the final grammar ballooned after all left side recursions, EBNF constructs and repetitions were removed, some constructs had to be split into smaller subsets to comply with LL(1) rules.

Section 2 - FIRST and FOLLOW sets:

I was able to generate the FIRST and FOLLOW sets of my newly LL(1) compliant grammar by using the grammartool that was given with the assignment handout. This was very useful because I was able to see exactly how my grammar was going to be used.

To simplify correcting errors and implementing the syntax into my project, I chose to create a map containing my entire grammar in the same format as EBNF but stored inside a C++ hashmap (std::unordered_map). Then at runtime I generate the FIRST and FOLLOW sets on the fly from my grammar with a simple algorithm, then use these to generate my parse table. Ideally these generations would be done at compile time instead of every time the program is run.

I can then use the FOLLOW set and parse table to generate my SYNC flags for my parser later down the road.

Section 3 - Design:

I used the table driven top-down predictive parser approach. Using my parse table I am able to iterate over the token list generated by my lexer in the previous assignment and using a stack system I can add all expected terminals and non-terminals for each non-terminal in my grammar. This is very powerful for error recovery since when a token that was expected is not found but there was only 1 possible choice, we can insert this token ourselves and repair the compilation without resulting in an error and preventing the build.

There is a limit in place for this error correction of a maximum of 25 warnings (recoverable errors) per file after which the parsing will error out and stop.

Section 4 - Use of tools:

I chose regular expressions for the lexical analyzer because they model lexing as a mathematical operation. A regular expression precisely defines the language of valid tokens, and when it is written correctly, the matching process has no side effects or ambiguity. This makes tokenization deterministic and predictable, reducing the risk of unexpected behavior compared to ad-hoc parsing logic.

With that said, I wrote the project using c++ and the standard library offers regex matching (std::regex). This worked out of the box but was incredibly slow. It would "lex" a file of around 100 lines containing around 75 tokens in around 10 milliseconds and when tested on a file with more than a million lines and approx 1.5M tokens it would spin forever (DNF).

This led me to search for 3rd party libraries that would still use Regular Expressions but in a more optimized approach. This is when I found "ctre", it is a c++ library that implements Regular Expressions at compile time instead of runtime. After implementing this library and optimizing my code at diverse places (mainly memory optimizations such as using file size to estimate final token amount and reserving space all at once instead of allocating more memory on the heap every time a token is generated), the same tests as before went down to ~0.05 milliseconds for the simple test and ~500 milliseconds (or half a second) for the 1.5M token test.

The logging library "spdlog" was used for logging asynchronously with levels. The build tool used "xmake" makes it very simple to include those 2 libraries.

Performance test on a file with around 6 million tokens (file output disabled):

```
(kapsulon|~/COMP442(git*main))✓> ./bin/syntactic-analyzer assignments/assignment2/tests/test_6M.src
[info   ] assignments/assignment2/tests/test_6M.src: Lexed 5982944 tokens in 755.56ms
[info   ] assignments/assignment2/tests/test_6M.src: Parsed in 774.68ms with 0 syntax error(s)
```